method.

#3 If this value is 0, meaning that the value of the coordinate compared is the same, then we go left half of the times, and right the other half. Otherwise, we just return the sign, that will be either 1 or -1.

This helps achieving a better-balanced tree on average, but still doesn't provide us any guarantee.

At date, there is not a solution to easily keep a k-d tree balanced while maintaining `O(h)` running time for insertion.

Nevertheless, if we know the set of points to insert in advance (when we create the k-d tree), then we can find an optimal order of insertion that allows us to construct a balanced tree. If the tree is not changed after construction, or if the number of elements inserted/deleted is negligible in comparison to the size of the tree, then we can have a worst-case balanced k-d tree, and `insert` and `search` will run in worst-case logarithmic time.

Listing 9.6 Balanced construction

```
function constructKdTree(points,
level=0)                                        #1
  if size(points) == 0
then                                                    #2
    return null
  elsif size(points) == 1
then                                                #3
    return new KdNode(points[0], null, null, level)
  else
    (median, left, right) ← partition(points,
level)            #4
```